

Voice Assistant + Lead Automation System

Yuktimantra Services

Complete Build Notes — n8n AI Automation Project

May 2026 | Built by Yash Shah

1. Project Overview

This document covers the complete build of a Voice Assistant and Lead Automation System for Yuktimantra Services. The system captures leads from the website, classifies them using AI, searches a knowledge base using RAG (Retrieval Augmented Generation), and automates follow-up via WhatsApp, Email, and Twilio calls.

1.1 What Was Built

- Voice assistant frontend (HTML + Web Speech API) — captures customer queries
- n8n automation workflow with 9+ nodes for complete lead processing
- RAG knowledge base using OpenAI Embeddings + Pinecone vector database
- AI-powered interest classification (High / Medium / Low) using GPT-4o
- Google Sheets integration for permanent lead storage
- Switch node routing leads to different automated follow-up branches

1.2 Tech Stack

Component	Technology Used
Frontend	HTML + Web Speech API (Voice input from browser)
Automation	n8n (self-hosted on localhost:5678)
AI Engine	OpenAI GPT-4o (lead classification + RAG answers)
Embeddings	OpenAI text-embedding-3-small (1536 dimensions)
Vector DB	Pinecone (yukthimantra-knowledge index)

Database	Google Sheets (Yuktimantra Leads spreadsheet)
Calling	Twilio (AI calls — Monday access pending)
WhatsApp	360Dialog / Twilio WhatsApp (pending)
Email	Gmail node in n8n (pending)
Language	JavaScript (n8n Code nodes), Python (Pinecone setup)

2. System Architecture

2.1 n8n Workflow — Complete Node Sequence

#	Node	Status	What it does
1	Webhook (Trigger)	Done	Listens for POST at /lead-intake. Receives lead data from frontend.
2	Edit Fields (Set)	Done	Maps full_name, phone_number, service_interest to clean field names.
3	Append Row in Sheet	Done	Saves lead row to Google Sheets Yuktimantra Leads.
4	HTTP Request (OpenAI Embed)	Done	Converts requirement to 1536-dim vector via text-embedding-3-small.
5	HTTP Request (Pinecone)	Done	Queries Pinecone to find top 3 relevant knowledge chunks.
6	Code Node (RAG Extract)	Done	Extracts text from matches, builds context string for GPT-4o.
7	HTTP Request (OpenAI GPT)	Done	Classifies lead as high/medium/low using requirement + context.
8	Edit Fields1	Done	Extracts interest_level from OpenAI response.
9	Switch Node	Done	Routes: high=Twilio, medium=WhatsApp, low=Email.
10	Twilio AI Call	Pending	Monday — AI calls high interest leads automatically.
11	WhatsApp Message	Pending	Sends WhatsApp intro to medium interest leads via 360Dialog.
12	Gmail Email	Pending	Sends info email to low interest leads.

3. RAG Implementation — Step by Step

3.1 What is RAG?

RAG = Retrieval Augmented Generation. Instead of AI answering from training data only, it first searches Yuktimantra's knowledge base and uses that info to give accurate, company-specific

answers.

3.2 RAG Flow Diagram

Part 1: Knowledge Base Setup (One Time)	Part 2: Query Flow (Every Lead)
1. Scraped Yuktimantra website content 2. Split into 500-char chunks 3. Converted to 1536-dim vectors (OpenAI) 4. Stored 9 vectors in Pinecone index 5. Index: yukthimantra-knowledge	1. Customer submits requirement 2. Requirement converted to vector 3. Pinecone finds top 3 similar chunks 4. Context + query sent to GPT-4o 5. AI classifies: high/medium/low

3.3 Pinecone Setup Details

- Index name: yukthimantra-knowledge
- Dimension: 1536 (matches text-embedding-3-small)
- Metric: cosine similarity
- Cloud: AWS us-east-1
- Total vectors uploaded: 9 chunks

3.4 Python Script to Upload Vectors

n8n network blocked Pinecone directly, so Python was used:

```
pip install pinecone openai
from pinecone import Pinecone
import openai
pc = Pinecone(api_key='YOUR_PINECONE_KEY')
index = pc.Index('yukthimantra-knowledge')
client = openai.OpenAI(api_key='YOUR_OPENAI_KEY')
response = client.embeddings.create(model='text-embedding-3-small', input=text)
embedding = response.data[0].embedding
index.upsert(vectors=[{'id': 'chunk-0', 'values': embedding, 'metadata': {'text': text}}])
```

3.5 Knowledge Base Content (9 Chunks Uploaded)

- Chunk 0: General overview — EdTech, staffing, outsourcing
- Chunk 1: EdTech Solutions — AI training, e-learning
- Chunk 2: Staffing Solutions — Recruitment, HR
- Chunk 3: Business Outsourcing — BPO, data science, medical coding
- Chunk 4: AI Automation — n8n workflows, AI agents
- Chunk 5: ERP Solutions — Billing management, enterprise software
- Chunk 6: Data Science — Analytics, ML, BI

- Chunk 7: Contact info — www.yuktimantraservices.com
 - Chunk 8: Additional service details
-

4. Errors Encountered & How We Fixed Them

Complete log of every error during the build with root cause and fix:

Error	Root Cause	Fix Applied
Webhook: 404 Not Found	Browser sends GET, webhook needs POST	Used Postman to send POST request properly
Webhook: not registered	Space in path: /webhook-test/ lead-intake	Deleted and retyped 'lead-intake' without spaces
Edit Fields: name = [null]	Form sends full_name but node expected name	Changed to {{\$json.body.full_name}} etc.
Google Sheets: empty	Headers typed vertically in col A not row 1	Cleared all, typed headers horizontally in Row 1
Google Sheets: Forbidden	Sheets API not enabled in Google Cloud	Enabled API at console.developers.google.com
requirement = undefined	Sheets node used body.service_interest directly	Changed to {{\$json.requirement}} from Set node
Pinecone: dim 0 != 1024	Index dimension 1024 but model gives 1536	Deleted index, created new with Custom Settings dim=1536
Pinecone: 409 already exists	Old index not deleted before recreating	Deleted old index from dashboard first
Pinecone: connection error	n8n local network blocked pinecone.io	Used Python script to upload vectors directly
OpenAI: submit an input	Input field had {{{ }}} triple braces	Cleared and retyped with exactly {{ }} double braces
content got null	requirement not passed through Code node	Added \$('Edit Fields').first().json.requirement fallback
JSON: bad control char	\n newlines in JSON body string literal	Used JSON.stringify() with expression mode
messages: got string	Messages array evaluated as string	Switched to Using JSON + JSON.stringify() wrapping

5. Node Configuration Details

Node 1 — Webhook

- HTTP Method: POST
- Path: lead-intake
- Respond: When Last Node Finishes
- Test URL: http://localhost:5678/webhook-test/lead-intake

Node 2 — Edit Fields

- name = {{\$json.body.full_name}}
- phone = {{\$json.body.phone_number}}
- requirement = {{\$json.body.service_interest}}
- source = yuktimantra_website

- status = new
- timestamp = {{\$now.toISO()}}

Node 3 — Google Sheets

- Resource: Sheet Within Document
- Operation: Append Row
- Document: Yuktimantra Leads
- Sheet: Sheet1 | Mapping: Map Each Column Manually

Node 4 — OpenAI Embeddings

- POST https://api.openai.com/v1/embeddings
- model = text-embedding-3-small
- input = {{\$json.requirement}}

Node 5 — Pinecone Query

- POST https://yukthimantra-knowledge-p59iobe.svc.aped-4627-b74a.pinecone.io/query
- topK = 3 | includeMetadata = true
- vector = {{\$json.data[0].embedding}}

Node 6 — Code Node (RAG Extract)

```
const matches = $input.first().json.matches;
const context = matches.map(m => m.metadata.text).join(' ');
const requirement = $input.first().json.requirement
|| $('Append row in sheet').first().json.requirement;
return [{ json: { context, requirement, name, phone } }];
```

Node 7 — OpenAI GPT-4o Classification

- POST https://api.openai.com/v1/chat/completions
- model = gpt-4o
- System: Classify as high medium or low. One word only.
- User: {{\$json.requirement}}

Node 8 — Edit Fields1

- interest_level = {{\$json.choices[0].message.content.trim().toLowerCase()}}

Node 9 — Switch

- Rule 1 (Output 0): interest_level = 'high' → Twilio call
- Rule 2 (Output 1): interest_level = 'medium' → WhatsApp
- Rule 3 (Output 2): interest_level = 'low' → Email

6. Progress Summary

Feature	Status	%
Webhook + Lead Capture	Done	100%
Edit Fields (Data Format)	Done	100%
Google Sheets Storage	Mostly Done	80%
OpenAI Embeddings (RAG)	Done	100%
Pinecone Vector DB	Done	100%
RAG Context Extraction	Done	100%
AI Interest Classification	Done	100%
Switch Node Routing	Done	100%
Twilio AI Calling	Pending (Monday)	0%
WhatsApp Messaging	Pending	0%
Email Follow-up	Pending	0%
Final Status Update	Pending	0%

Overall Project Completion: 65%

Prepared by Yash Shah | Yuktimantra Services | May 2026

Next session: Monday — Twilio + WhatsApp + Email branches